HACK-TO-WEEN 2026 / OPEN INNOVATION

GhostHire

Expose fake opportunities before they ghost you.

An evidence-based opportunity verification platform for students evaluating internships, jobs, scholarships, hackathons and other online offers.

> THE CORE IDEA
> Turn suspicious messages and screenshots into structured, explainable risk reports - with practical next steps and community pattern intelligence.

PROJECT CONCEPT • MVP PLAN • AWS ARCHITECTURE • DEMO SCRIPT

Prepared as a build-ready hackathon project brief. The examples and risk scores in this document are illustrative, not validated fraud probabilities.

01 / Executive summary

GhostHire helps students assess suspicious opportunities before they pay money, share credentials or submit sensitive information. A user can paste a message or upload a screenshot. GhostHire extracts the content, checks for risk indicators, and returns an explainable report with evidence and recommended verification steps.

> **The product promise**
> Not “AI says this is a scam.” Instead: “Here are the specific signals we found, the evidence behind each one, and how you can independently verify the opportunity.”

The problem

Students regularly encounter internship, job, scholarship and event announcements through social media, messaging apps and unofficial websites. Some messages imitate trusted organizations, demand upfront fees, use lookalike domains or create pressure to act immediately. The student is left to investigate each claim manually.

Target users

- Primary: college students and early-career applicants who receive opportunities through WhatsApp, email, social media and job portals.
- Secondary: placement cells, student clubs and campus communities that want to share safety guidance and patterns of reported suspicious domains.

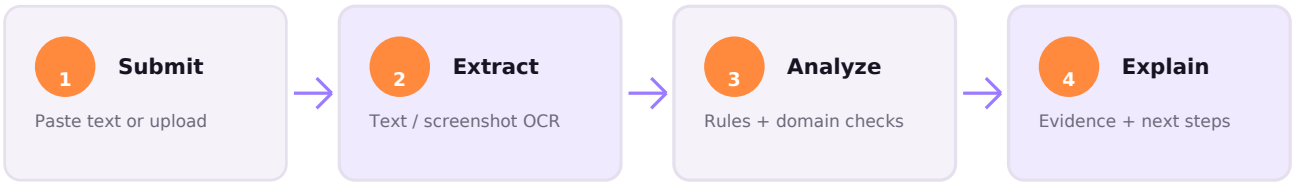
Why the idea is worth building

- The problem is relatable and easy to demonstrate with a realistic sample message.
- The product has a clear input and output: message or image in; evidence-based report out.
- AWS services perform real work in the core flow rather than appearing as superficial integration.
- The same workflow can expand to more languages, opportunity types and institutional verification sources.

> **Scope guardrail**
> GhostHire should flag risk indicators, not declare a company or person guilty. A risk score is a prioritization aid, not proof of fraud.

02 / Product experience

A compact workflow makes the MVP easy to understand and easy to test.



Core user journey

1. Submit an opportunity. The user pastes a message or uploads an image containing an internship, job or scholarship offer.

2. Extract the information. Text is parsed directly. Images are stored temporarily and sent through OCR to retrieve their text.

3. Analyze the evidence. A Python rules engine looks for payment requests, urgency, guaranteed-selection claims, suspicious URLs, domain mismatches and credential requests.

4. Show an explainable report. Findings include the signal, exact evidence, severity and a safe next step.

5. Check community intelligence. With user consent, the system can compare a domain or pattern against previously submitted reports.

MVP feature priorities

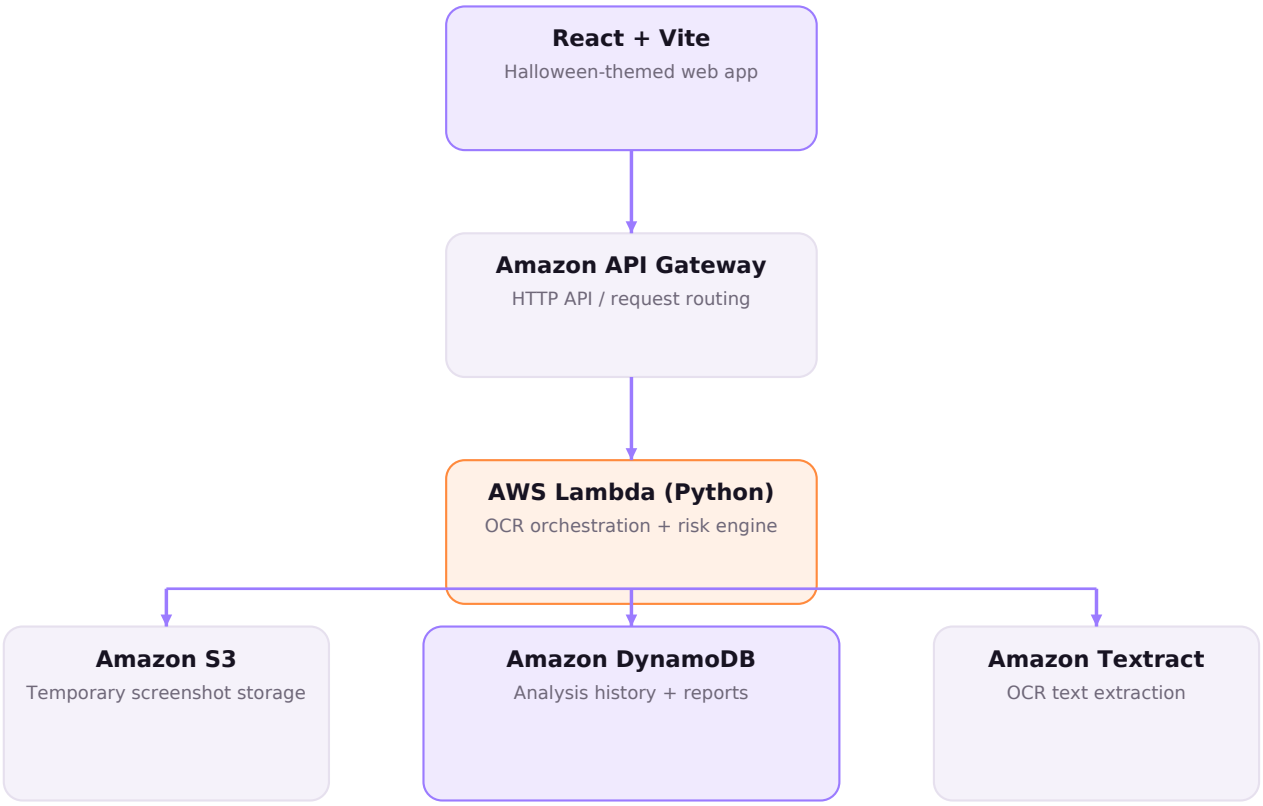
FEATURE	MVP BEHAVIOR	PRIORITY
Message scanner	Accept pasted text and analyze it with the risk engine.	**Must have**
Screenshot scanner	Upload image, extract text using Textract, pass text into the same engine.	**Must have**
Explainable report	Display score, risk label, evidence snippets and recommendations.	**Must have**
Analysis history	Store the result and metadata for later viewing.	**Should have**
Community reports	Search user-submitted domain/pattern reports, with consent and moderation.	**Should have**

Build decision

Do not build every feature. The strongest hackathon MVP is screenshot upload → OCR → risk analysis → evidence report. Add community history only after this end-to-end path is reliable.

03 / AWS architecture

Recommended architecture: React + Vite on the frontend, with serverless Python services on AWS. The diagram shows the intended MVP data path.



Upload path: browser -> presigned S3 URL -> Textract -> Lambda analysis -> DynamoDB result

What each AWS service does

SERVICE	RESPONSIBILITY IN GHOSTHIRE
Amazon API Gateway	Receives frontend requests and routes them to backend functions.
AWS Lambda (Python)	Runs the risk engine, coordinates OCR, validates requests and formats reports.
Amazon S3	Temporarily stores uploaded screenshots; use short-lived objects and lifecycle cleanup.
Amazon Textract	Extracts readable text from uploaded screenshots or documents.
Amazon DynamoDB	Stores analysis metadata, findings and community reports.

Implementation and security notes

- Use React/Vite/Tailwind in the browser and Python Lambda behind API Gateway; host the frontend on Vercel if convenient.
- Never expose AWS secrets in browser code. Use Lambda execution roles and presigned S3 upload URLs.
- Validate file type, file size and request schema; rate-limit public endpoints.
- Delete temporary screenshots after processing or enforce a short S3 lifecycle policy. Store raw content only when necessary and with consent.

04 / Explainable risk engine

The risk engine is the central product differentiator. It should produce a traceable score from explicit signals, then show the evidence for every signal that contributed.

Illustrative scoring model

SIGNAL	EXAMPLE EVIDENCE	POINTS
Upfront payment request	“Pay ₹999 registration fee before selection.”	**+25**
Guaranteed selection	“Selection is guaranteed.”	**+15**
Sender/domain mismatch	Sender domain does not match the independently verified organization domain.	**+20**
Suspicious or shortened URL	Shortened link or destination with suspicious characteristics.	**+15**
Artificial urgency	“Reply within 30 minutes or lose your seat.”	**+10**
Credential/financial data request	Requests OTP, password or sensitive financial credentials.	**+15**

Example report score

65 / 100 **HIGH RISK - illustrative example**



Proposed labels: Low 0–29 • Medium 30–59 • High 60–100. These thresholds are prototype settings and should be tested against labeled examples before making real-world claims.

Required structure for every finding

- signal: a stable machine-readable identifier, such as UPFRONT_PAYMENT.
- evidence: the exact text, domain or attribute that triggered the signal.
- points: the contribution to the illustrative score.
- recommendation: a safe and practical action for the user.

Guardrails against misleading results

- Do not treat the score as a statistically calibrated probability of fraud.
- Do not infer a domain mismatch unless the organization domain is independently verified.
- Treat missing information as unknown, not as proof that an opportunity is safe or unsafe.
- Clearly distinguish an observed signal from a confirmed fact.
- Provide a path to correct or dispute a community report; moderate reports before wider display.

05 / API design and live demo

Keep the backend surface small so the team can finish integration and testing under hackathon time pressure.

Suggested API endpoints

METHOD + PATH	PURPOSE
POST /analyze	Accept text or a reference to an uploaded image; return a structured risk report.
POST /report	Submit a domain or pattern report with user consent and moderation metadata.
GET /community/{domain}	Return approved, aggregated reports for a domain.

Example response shape

```
{
"risk_score": 65,
"risk_level": "HIGH",
"findings": [
{"signal": "UPFRONT_PAYMENT", "points": 25,
"evidence": "Pay ₹999 registration fee"},
{"signal": "GUARANTEED_SELECTION", "points": 15,
"evidence": "Selection is guaranteed"},
{"signal": "SUSPICIOUS_URL", "points": 15,
"evidence": "bit.ly/example"},
{"signal": "ARTIFICIAL_URGENCY", "points": 10,
"evidence": "Reply within 30 minutes"}
],
"recommendation": "Verify via the official careers website."
}
```

Live demo script

Use a fabricated sample opportunity so the demo does not make an unsupported allegation about a real organization:

> **Demo input**
>
> “Congratulations! You have been shortlisted for the Nexora Labs internship. Pay ₹999 registration fees today. Selection is guaranteed. Apply through bit.ly/example. Reply within 30 minutes to secure your seat.”

Expected demo result

- Upfront payment: +25; guaranteed selection: +15; shortened URL: +15; artificial urgency: +10.
- Illustrative total: 65/100, classified as High under the proposed prototype thresholds.
- Recommendation: do not pay; independently find the organization’s official website and verify the application channel.
- Repeat the exact same scenario with a screenshot to demonstrate S3 + Textract + the shared analysis pipeline.

06 / UI direction and presentation

The submission explicitly requires a Halloween-themed UI. The visual theme should make the project memorable without making risk information hard to read.

Design system

ROLE	COLOR	HEX
Background	Deep near-black purple	#100D19
Cards	Dark plum	#211A2D
Primary accent	Halloween orange	#FF8A3D
Secondary accent	Electric lavender	#9B7BFF
Danger	Coral red	#FF5B66

UI elements to include

- Ghost-radar inspired dashboard with a clear “scan opportunity” primary action.
- Drag-and-drop upload area for screenshots; paste-text input for messages.
- Risk result card with score, label and short explanation.
- Evidence list with highlighted phrases and a recommended next step.
- Subtle floating ghost motifs, scanning animation and Halloween iconography - avoid excessive animation.

Seven-slide pitch deck outline

SLIDE	CONTENT
1. Title	GhostHire - Expose Fake Opportunities Before They Ghost You.
2. Problem	Fake internship, job, scholarship and event offers; students lack fast verification.
3. Solution	Paste or upload → extract → analyze → explain → verify.
4. Product demo	Show screenshot upload and the resulting evidence-based report.
5. Architecture	React, API Gateway, Lambda, S3, Textract and DynamoDB.
6. Innovation + impact	Explainability, privacy-conscious community intelligence and student safety.
7. Future scope	More languages, more opportunity types and verified institutional sources.

07 / Build plan and judging strategy

Prioritize a working end-to-end path over feature count. For the November 5, 2026 deadline, use this focused same-day build plan.

Suggested build schedule (IST)

TIME	DELIVERABLE
5:20–6:10 PM	React frontend shell and Halloween visual system.
6:10–7:10 PM	Python risk engine with unit tests and sample messages.
7:10–8:20 PM	S3 upload path and Textract extraction.
8:20–9:10 PM	Lambda, API Gateway and DynamoDB integration.
9:10–10:00 PM	Connect frontend to backend; test text and screenshot flows.
10:00–10:40 PM	Polish dashboard, seed safe demo data and fix defects.
10:40–11:20 PM	Prepare 7-slide PPT and capture the demo.
11:20–11:50 PM	Final testing and submit through Unstop before 11:59 PM.

How to defend the idea to judges

- “Is it just an AI chatbot?” No. The core is a structured rules engine with OCR and evidence extraction; an LLM is optional.
- “Can you prove fraud?” No. GhostHire flags risk signals and recommends independent checks; it does not make definitive fraud determinations.
- “What is innovative, and why AWS?” Screenshot OCR, explainable signals and community patterns are delivered through real AWS storage, OCR, APIs and database services.

Judge-facing success criteria

- Text and screenshot submissions both reach the same structured report flow.
- Every score contribution has visible evidence; AWS integration works live without frontend secrets.
- The interface follows the Halloween theme while keeping warnings readable.

> **“Every fake opportunity begins with a message. Every student deserves a way to question it before trusting it. GhostHire turns suspicious messages into actionable evidence, helping students make safer decisions in the digital world.”**

Final recommendation

Prioritize three things: (1) a polished, usable frontend; (2) real screenshot-to-Textract-to-risk-report functionality; and (3) an explainable risk engine with AWS-backed report storage. A small, reliable MVP will make a stronger submission than a large unfinished feature set.

Disclaimer: This brief is a product proposal, not a guarantee of detection accuracy. Validate the scoring model with representative examples, protect user data, and avoid presenting unverified community allegations as facts.